About Trainer

- Ms. Smita Kadam
  - **Associate Director** at Sunbeam
  - MCM from Pune University
- Training and Management
  - PreCAT
- Contact
  - Email: smita@sunbeaminfo.com
  - Mobile: 9373084868

SUNBEAM

Data Structure

- Section B
  - Questions 10
- Syllabus
  - Searching
  - Sorting
  - Stack Operations/Implementations & Applications
  - Queue Operations/Implementations
  - Linked List
  - Tree and Graph Terminologies
  - Hash Table

SUNBEAM

Introduction to Data Structure

What is Algorithm?

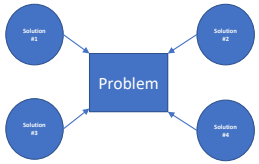
Logical flow for given problem statement. Can be represented in two ways :

1. Pseudocode

   Logical flow written in simple English language.
2. Flowchart

   Logical flow represented in pictorial format.

Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.



SUNBEAM

Data Structure

Algorithm can be categorised for different actions as below :

1. **Searching**
2. **Sorting (Ascending / Descending)**
3. **Inserting a Element**
4. **Update a Element**
5. **Delete a Element**

Advantages of Algorithm:

1. **Uses definite procedure**
2. **Logical steps makes debug process easy**
3. **Smaller problems if divided into parts can be easily solved**

Disadvantages of Algorithm:

1. **Algorithm can be time consuming.**
2. **Each algorithm can not be divided into multiple small algorithms hence can be difficult.**
3. **It becomes difficult to represent iterative or jumping into another small algorithm**

SUNBEAM

Data Structure

Types of Algorithm:

1. **Recursive Algorithm**
   - **A algorithm which calls itself repeatedly**
   - **In each recursive call is done with smaller value which makes it easy to solve**
2. **Divide and Conquer Algorithm** binary search, merge sort, quick sort
   - **Divide: This step divides the problem into one or more substances of the same problem of smaller size**
   - **Conquer: Provides solutions to the bigger problem by using the solutions of the smaller problem by some additional work**
3. **Greedy Algorithm**
   - **Greedy algorithms are generally used in optimization problems by taking decision at each step.**
   - **There is an optimal substructure to the problem.**
   - **Try to maximize the function by making greedy choice at all points.**
   - **Without planning ahead ignores future possibilities**
   - **No backtracking. Usually O(n) or O(n log n) Solutions.**

Data Structure

Types of Algorithm:

1. **Brute Force Algorithm**
   - The brute force solution is simply to calculate the total distance for every possible route and then select the shortest one. This is not particularly efficient because it is possible to eliminate many possible routes through clever algorithms.
2. **Back Tracking Algorithm:**
   - **Decision Problem – searches for feasible solution**
   - **Optimization Problem- searches best solution**
   - **Enumeration Problem – all feasible solutions**
   - **A backtracking algorithm uses the depth-first-search method. When it starts exploring the solutions, a bounding function is applied so that the algorithm can check if the so-far built solution satisfies the constraints. If it does, it continues searching. If it doesn't, the branch would be eliminated, and the algorithm goes back to the level before.**
3. **In Place Algorithm**
   - *In-place means that the algorithm does not use extra space for manipulating the input but may require a small though non-constant extra space for its operation. Usually, this space is O(log n), though sometimes anything in O(n) (Smaller than linear) is allowed.*

Data Structure

Types of Algorithm:

4. Dynamic Programming Algorithm (DP)

- DP is another technique for problems with optimal substructure.
- An optimal solution to a problem contains optimal solutions to sub-problems.
- This doesn't necessarily mean that every optimal solution to a sub-problem will contribute to the main solution.
- For divide and conquer (top down), the sub problems are independent so we can solve them in any order.
- In dynamic programming, we solve many sub-problems and store the results
- For greedy algorithms (bottom up), we can always choose the "right" sub-problem by a greedy choice.
- not all of them will contribute to solving the larger problem.
- Because of optimal substructure, we can be sure that at least some of the sub-problems will be useful

Efficiency of an algorithm

for(i = 1; i<=10; i++)
{
}

- Efficiency of a algorithm can be concluded based on
  - Time (Running Time)
  - Space
- Time Complexity for a given algorithm can be represented after doing asymptotic or symptotic analysis. Time is calculated can change depends on system configuration.
  - **Asymptotic Analysis** – Approximate time span required to process an algorithm. Generally common functionality is considered. e.g. count of comparisons or iteration count.
  - **Symptotic Analysis** – This method calculates time required to execute given process. It will prove exact time used by a process.
- **Notations used to represent asymptotic time complexity**:
  - **Big-oh Notation (O)** - To express an upper bound on the time complexity as a function of the input size. It also captures worst case analysis of an algorithm.
  - **Omega (Ω)** - To express a lower bound on the time complexity as a function of the input size. It also denotes the best case analysis of an algorithm.
  - **Theta (Θ)** - To express the tight bound on the time complexity as a function of the input size. It can be used to denote tight bounds of an algorithm. i.e., g(n) is a lower bound as well as an upper bound.

Types of Time Complexity

- **Constant Time Complexity**
  - Always process data in fixed time span no mater what is the count of input values less or large volume.
  - The most efficient type of time complexity, however achieving constant time complexity is not usually possible due to the nature of computation and is typically associated with simple operations or data structures that can be accessed directly in a single step.
- **Linear Time Complexity**
  - Linear time is when the runtime scales linearly as the input grows.
  - So if the input is 1 number, the runtime will be 1 step. If the input is 1 million numbers, the runtime will be 1 million steps.
  - The most common type of complexity. It is fairly efficient as it's a one to one growth.
  - It is more efficient than quadratic time complexity.
- **Quadratic Time Complexity**
  - Quadratic time is when the runtime scales quadratically as the input grows.
  - If the input size is 1, the runtime is 1 step. If the input size is 10, the runtime is 100 steps. If the input size is 100, the runtime is 10,000 steps.(square of input size)
  - Runtime is growing at a much faster rate compared to linear growth.
  - Not very efficient. if possible avoid use of this.
- **Logarithmic Time Complexity**
  - Max case base is considered as 2 unless it is not specified with required value
  - Logarithmic time is when the input size increases, the time taken by the algorithm increases.
  - Runtime is slow rate compared to linear or quadratic growth.
  - If we have 2 to the power of 3, the logarithm is 3. If we have 2 to the power of 4, the logarithm is 4. If we have 2 to the power of 5, the logarithm is 5. And so on.

SUNBEAM

Calculate Time Complexity -The most common metric it's using Big O notation.

Source Code variable	Time Complexity

int n=5;
int k = n * n;
printf("%d",k);
	Constant Time Complexity can be represented as O(1)

n = 5;
for(int i=1;i<=n;i++) {
printf("value of i=%d \n",i);
}
	Linear Time Complexity
	Time will be calculated based on no. of iterations.
	Here iteration count is depend on n value.
	Hence time can be time $\propto$ n nothing but t(n) = O(n)

for(int i=0;i<n; i++) //1st loop
{
for(j=0; j<n; j++) //2nd loop
{
printf("value of j=%d \n", j);
}
}
	Quadratic Time Complexity
	Here 1st loop will iterate n times
	Even 2nd loop iterate n times for each value of i variable.
	We can consider it will consume n * n iterations nothing but n^2
	Hence time complexity can be O(n^2)
	Due to the nested loops, the time required to execute this code increases exponentially with the input size. As the array becomes larger, the execution time grows much faster, making it inefficient for larger inputs.

Calculate Time Complexity -The most common metric it's using Big O notation.

Source Code variable	Time Complexity

int main()
{
int arr[8] = {11,12,13,14,15,16,17,18};
int key = 16;
int low=0,high=7,mid,pos,flag=0;
while(low<=high)
{
mid = (low + high)/2;
if(key == arr[mid])
{
pos=mid; flag=1;
break;
}
else if (key < arr[mid])
high = mid - 1;
else if(key > arr[mid])
low = mid + 1;
}
if (flag==0)
pos=-1;
}
	Binary search works on **Divide and conquer** approach, In this example in worst case 3 comparisons will be needed to find any value in array. Also the value of log (N) where N is input size i.e. 8 for above example will be 3. Hence the algorithm can be said to exhibit logarithmic time complexity.

Efficiency of an algorithm

- Space Complexity:
  - Analyse space required by an algorithm
  - Space(Memory) can be analysed using factors
    - **Fixed Space:**
      - Depends on data type used to request memory.
      - Remains fixed for any situation do not matter what is size of input elements.)
    - **Variable Space**: Memory required for an algorithm depends on n elements at runtime.

SUNBEAM

Time Complexity Chart

Algorithm	Best Case	Average Case	Worst Case
Searching Algorithms			
Binary Search	$\Omega(1)$	$\Theta(n)$	$O(n)$
Linear Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$
Sorting Algorithms			
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Stack data structure			
Push	$\Omega(1)$	$\Theta(1)$	$O(1)$
Pop	$\Omega(1)$	$\Theta(1)$	$O(1)$
Peek	$\Omega(1)$	$\Theta(1)$	$O(1)$
Queue Data Structure			
Enqueue	$\Omega(1)$	$\Theta(1)$	$O(1)$
Dequeue	$\Omega(1)$	$\Theta(1)$	$O(1)$
Peek	$\Omega(1)$	$\Theta(1)$	$O(1)$
Display	$\Omega(1)$	$\Theta(1)$	$O(1)$

Time Complexity Chart

Algorithm	Best Case	Average Case	Worst Case
Singly Linear Linked List			
Add at first	$\Omega(1)$ if list is empty	$\Theta(1)$	$O(n)$
Add at last	$\Omega(1)$	$\Theta(n)$	$O(n)$
Add at given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$
Singly Circular Linked List			
Add at first	$\Omega(1)$	$\Theta(n)$	$O(n)$
Add at last	$\Omega(1)$ if list is empty	$\Theta(n)$	$O(n)$
Add at given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$
Doubly Linear Linked List			
Add at first	$\Omega(1)$	$\Theta(1)$	$O(1)$
Add at last	$\Omega(1)$ if list is empty	$\Theta(n)$	$O(n)$
Add at given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$
Doubly Circular Linked List			
Add at first	$\Omega(1)$	$\Theta(1)$	$O(1)$
Add at last	$\Omega(1)$ if list is empty	$\Theta(1)$	$O(1)$
Add at given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$

Time Complexity Chart

Algorithm	Best Case	Average Case	Worst Case
Singly Linear Linked List (if only head pointer is implemented)			
Delete from first	$\Omega(1)$ if list has single node	$\Theta(1)$	$O(1)$
Delete from last	$\Omega(1)$ if list has single node	$\Theta(n)$	$O(n)$
Delete from given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$
Singly Circular Linked List			
Delete from first	$\Omega(1)$ if list has single node	$\Theta(n)$	$O(n)$
Delete from last	$\Omega(1)$ if list has single node	$\Theta(n)$	$O(n)$
Delete from given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$
Doubly Linear Linked List			
Delete from first	$\Omega(1)$ if list has single node	$\Theta(1)$	$O(1)$
Delete from last	$\Omega(1)$ if list has single node	$\Theta(n)$	$O(n)$
Delete from given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$
Doubly Circular Linked List			
Delete from first	$\Omega(1)$ if list has single node	$\Theta(1)$	$O(1)$
Delete from last	$\Omega(1)$ if list has single node	$\Theta(1)$	$O(1)$
Delete from given position	$\Omega(1)$(if pos is 1)	$\Theta(n)$	$O(n)$

Space Complexity Chart

Algorithm	Best Case
Searching Algorithms	
Binary Search	O(1)
Linear Search	O(1)
Sorting Algorithms	
Selection Sort	O(1)
Bubble Sort	O(1)
Insertion Sort	O(1)
Merge Sort	O(n)
Quick Sort	O(log n)
Stack data structure	
Push	O(1)
Pop	O(1)
Peek	O(1)
Queue Data Structure	
Enqueue	O(1)
Dequeue	O(1)
Peek	O(1)
Display	O(1)

Algorithm	Best Case
Linked List	
Singly Linked List(Linear/Circular)	O(n)
Singly Linked List(Linear/Circular)	O(n)
Add operation of any Linked List	O(1)
Delete operation of any Linked List	O(1)
Traverse operation of any Linked List	O(1)